



LAPORAN PRAKTIKUM

Pengembangan Aplikasi Web II
Topik 32: Stasiun Pengisian Kendaraan Listrik

Disusun oleh:

Nama : Naufal Samih Najasyi

NIM : 2428240116

Kelas : SI5B

PROGRAM STUDI SISTEM INFORMASI
UNIVERSITAS MULTI DATA PALEMBANG
TAHUN AKADEMIK 2026/2027

Identitas Praktikum

Nama Mahasiswa	Naufal Samih Najasyi
NPM	2428240116
Kelas	SI5B
Pertemuan Ke-	5
Topik / Judul Praktikum	Stasiun Pengisian Kendaraan Listrik
Tanggal Pelaksanaan	5 Oktober 2026
Dosen Pengampu	Nur Rachmat, M.Kom.

I. Tujuan Praktikum

1.

Membangun RESTful API murni menggunakan Node.js dan Express.js untuk resource stasiun pengisian kendaraan listrik

2.

Mengimplementasikan enam endpoint RESTful dengan operasi GET, POST, PUT, DELETE, dan filter query string.

3.

Menerapkan validasi field wajib, respons JSON dan HTTP status code yang tepat, serta menguji API di lokal dan Vercel.

II. Dasar Teori

RESTful API adalah antarmuka layanan yang mengelola resource melalui URL dan method HTTP. Pada praktikum ini, resource `/charging-stations` mewakili data stasiun pengisian kendaraan listrik. GET digunakan untuk membaca data, POST menambahkan data, PUT mengganti data berdasarkan ID, sedangkan DELETE menghapus data. Express.js menyediakan routing dan middleware `express.json()` untuk membaca request body berformat JSON. Data disimpan dalam array di memori tanpa database. Respons GET sukses berupa objek atau array langsung, sementara respons POST, PUT, DELETE, dan error menggunakan struktur `{ status, message, data }`. Status HTTP yang digunakan antara lain 200, 201, 400, dan 404.

III. Alat dan Bahan

No.	Nama Alat / Software	Versi / Keterangan
1	Node.js	v24.21.0 (LTS)
2	Visual Studio Code	Editor kode
3	Express.js	^5.2.1
4	Nodemon	^3.1.14 (devDependency)
5	npm	v11.19.0
6	Thunder Client	Pengujian request/response API
7	Git dan GitHub	Git 2.56.0.windows.1; repository Public
8	Vercel	Deployment RESTful API

IV. Langkah Kerja

1.

Membuat project Node.js pada folder Tugas1_RESTful_2428240116, kemudian memasang Express.js dan Nodemon serta mengatur script start dan dev.

```
npm init -y | npm install express | npm install --save-dev nodemon
```

2.

Membuat app.js, memasang middleware express.json(), menyiapkan array dengan tiga data awal dan nextId untuk ID otomatis.

```
app.use(express.json()); // data awal disimpan dalam array di memori
```

3.

Membuat endpoint GET semua data, GET berdasarkan ID, POST, PUT, DELETE, dan filter berdasarkan query kota.

```
GET /charging-stations | POST /charging-stations | PUT /charging-stations/:id |  
DELETE /charging stations/:id
```

4.

Memvalidasi field wajib (nama, kota, jumlahSlot, dan dayaKw) serta memastikan seluruh respons sukses dan error berformat JSON.

```
res.status(400).json({ status: 'error', message: 'Field kota wajib diisi', data:  
null });
```

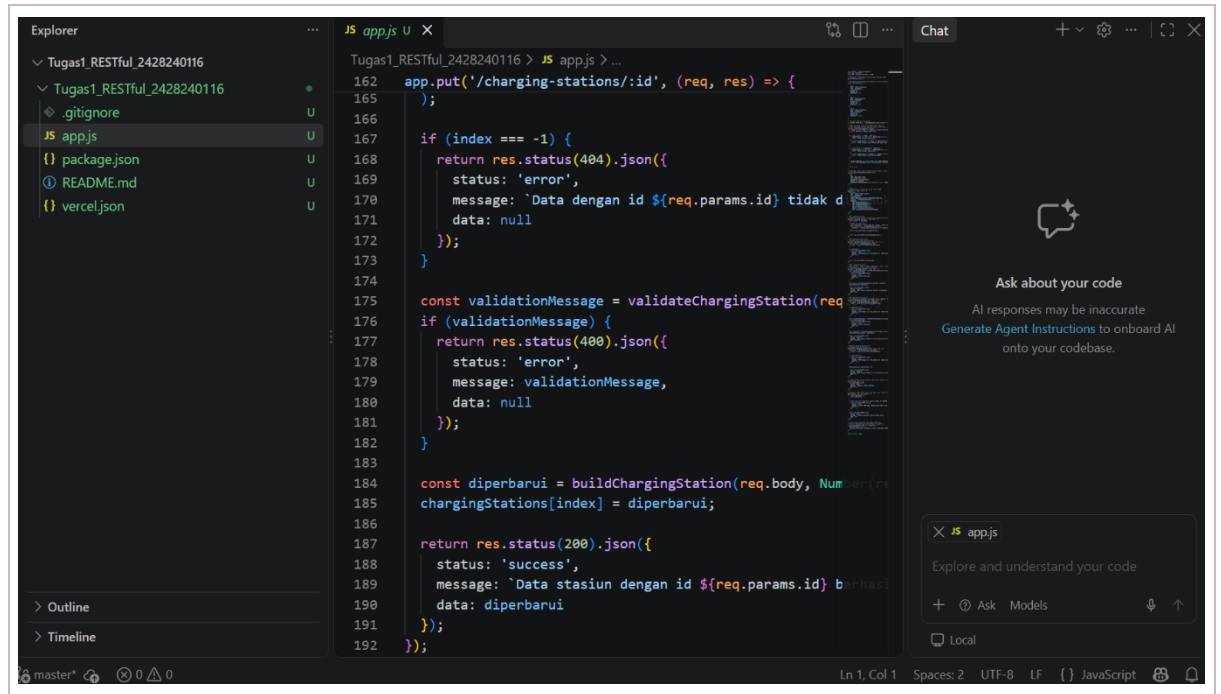
5.

Menguji endpoint dengan Thunder Client pada URL Vercel, mengunggah source code ke GitHub Public, dan memeriksa lima commit bertahap.

```
npm run dev | git push -u origin main | module.exports = app;
```

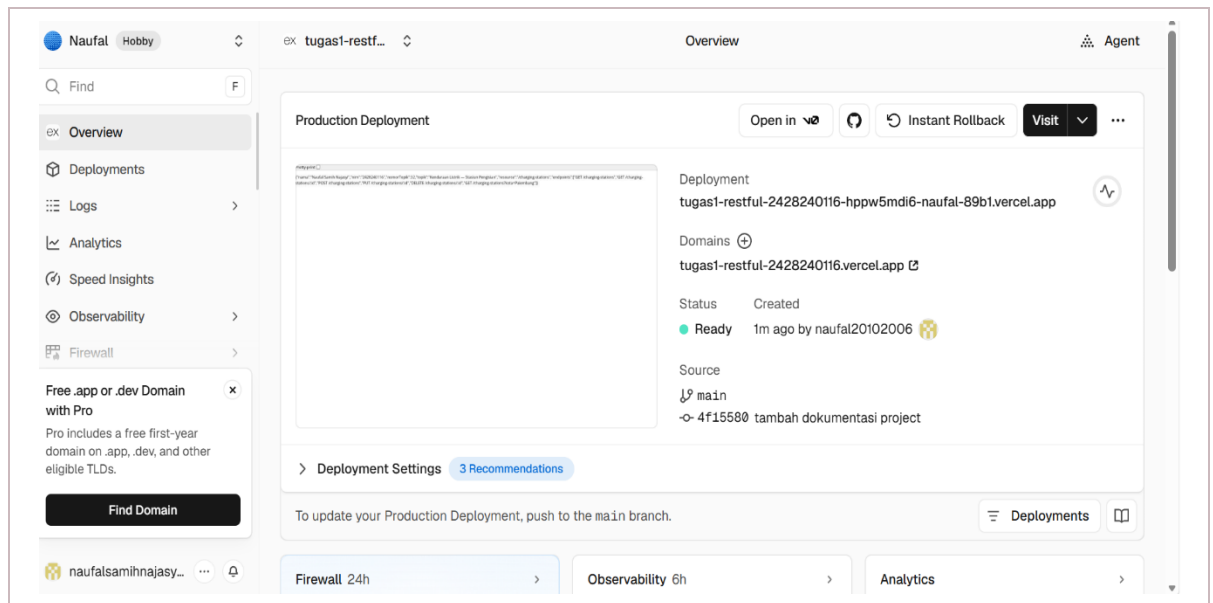
V. Hasil dan Pembahasan

Pengujian dilakukan melalui Thunder Client pada URL production Vercel. Tabel merangkum sepuluh skenario utama; setiap skenario diikuti bukti request, response JSON, dan status HTTP.



```
162 app.put('/charging-stations/:id', (req, res) => {
163   );
164 }
165
166 if (index === -1) {
167   return res.status(404).json({
168     status: 'error',
169     message: 'Data dengan id ${req.params.id} tidak d
170     data: null
171   });
172 }
173
174
175 const validationMessage = validateChargingStation(req
176 if (validationMessage) {
177   return res.status(400).json({
178     status: 'error',
179     message: validationMessage,
180     data: null
181   });
182 }
183
184 const diperbarui = buildChargingStation(req.body, Number(req
185 chargingStations[index] = diperbarui;
186
187 return res.status(200).json({
188   status: 'success',
189   message: 'Data stasiun dengan id ${req.params.id} berhasil
190   data: diperbarui
191 });
192 });
```

Gambar 1. Source code app.js yang memuat route PUT, validasi, dan respons JSON.



Gambar 2. Dashboard Vercel yang menampilkan production deployment berstatus Ready.

Pembahasan:

Hasil pengujian menunjukkan GET mengembalikan objek atau array langsung, sedangkan filter kota hanya mengembalikan stasiun yang sesuai. POST berhasil dengan status 201 dan menolak body tanpa kota menggunakan status 400.

PUT berhasil memperbarui resource dengan ID yang tersedia (200), dan menghasilkan 404 bila ID tidak ditemukan. DELETE mengembalikan 200 ketika data dihapus serta 404 untuk ID yang tidak ada. Seluruh respons yang diuji berupa JSON.

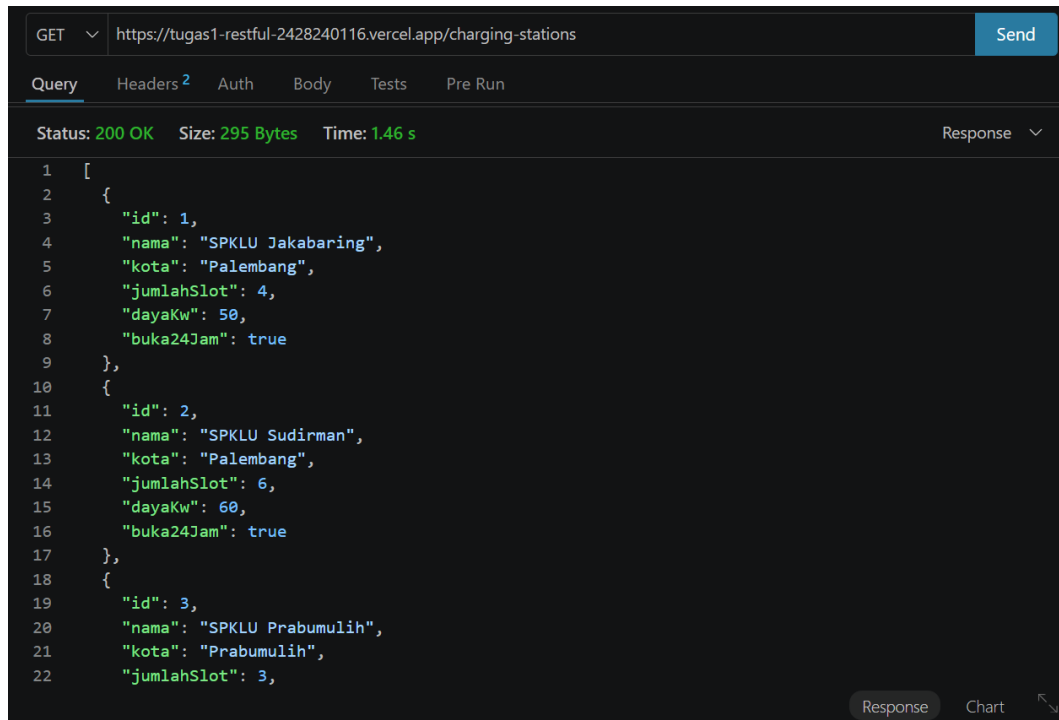
Pada Vercel, data menggunakan array di memori sehingga perubahan tidak dijamin bertahan di antara request serverless. Pengujian difokuskan pada kebenaran respons setiap request sesuai ketentuan tugas.

Ringkasan Hasil Pengujian

No .	Method / Endpoint	Data uji	Status target	Status hasil	Keterangan
1	GET /charging-stations	-	200	200	Array semua data
2	GET /charging-stations/1	-	200	200	Detail stasiun
3	GET /charging-stations/99	-	404	404	ID tidak ditemukan
4	GET /charging-stations?kota=Palembang	Kota Palembang	200	200	Filter kota
5	POST /charging-stations	Body lengkap	201	201	Data ditambahkan
6	POST /charging-stations	Tanpa kota	400	400	Validasi wajib
7	PUT /charging-stations/1	Body lengkap	200	200	Data diperbarui
8	PUT /charging-stations/99	Body lengkap	404	404	ID tidak ditemukan
9	DELETE /charging-stations/2	-	200	200	Data dihapus
10	DELETE /charging-stations/99	-	404	404	ID tidak ditemukan

Bukti Pengujian 1: GET /charging-stations

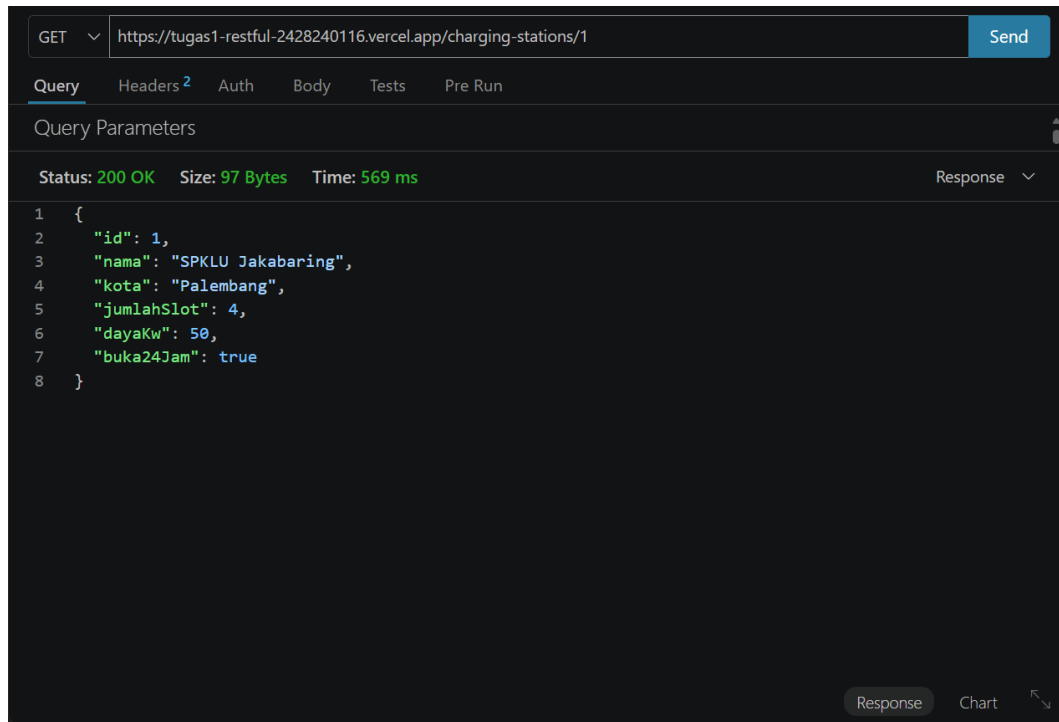
200 OK - seluruh data stasiun



Gambar 3. Bukti Thunder Client untuk GET /charging-stations; hasil 200 OK - seluruh data stasiun.

Bukti Pengujian 2: GET /charging-stations/1

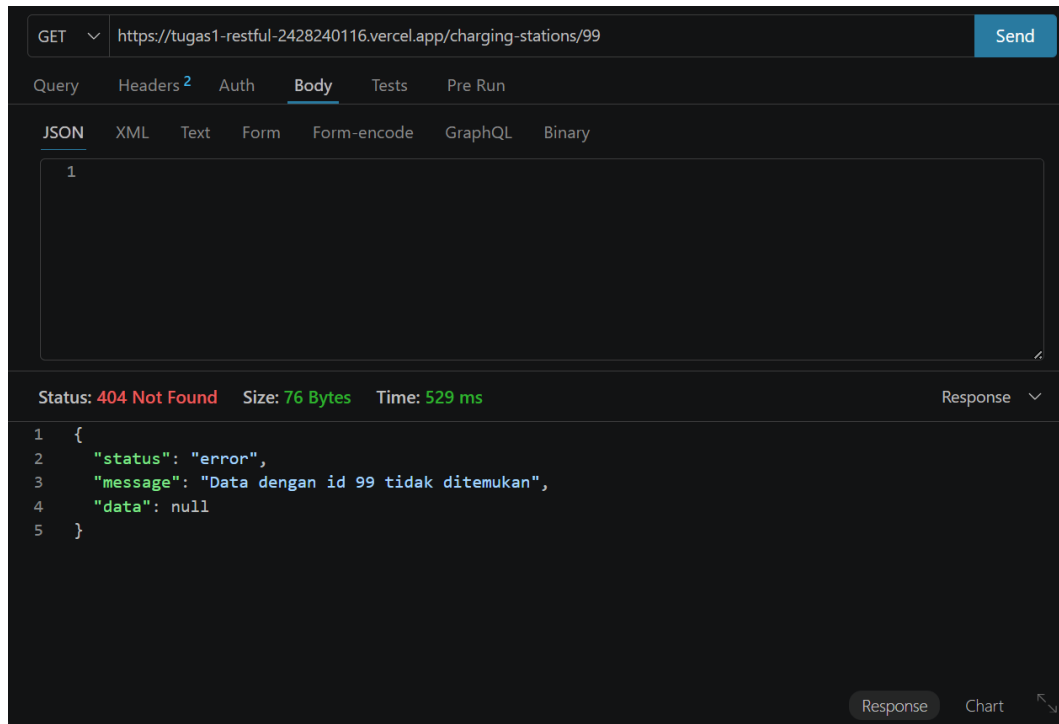
200 OK - detail SPKLU Jakabaring



Gambar 4. Bukti Thunder Client untuk GET /charging-stations/1; hasil 200 OK - detail SPKLU Jakabaring.

Bukti Pengujian 3: GET /charging-stations/99

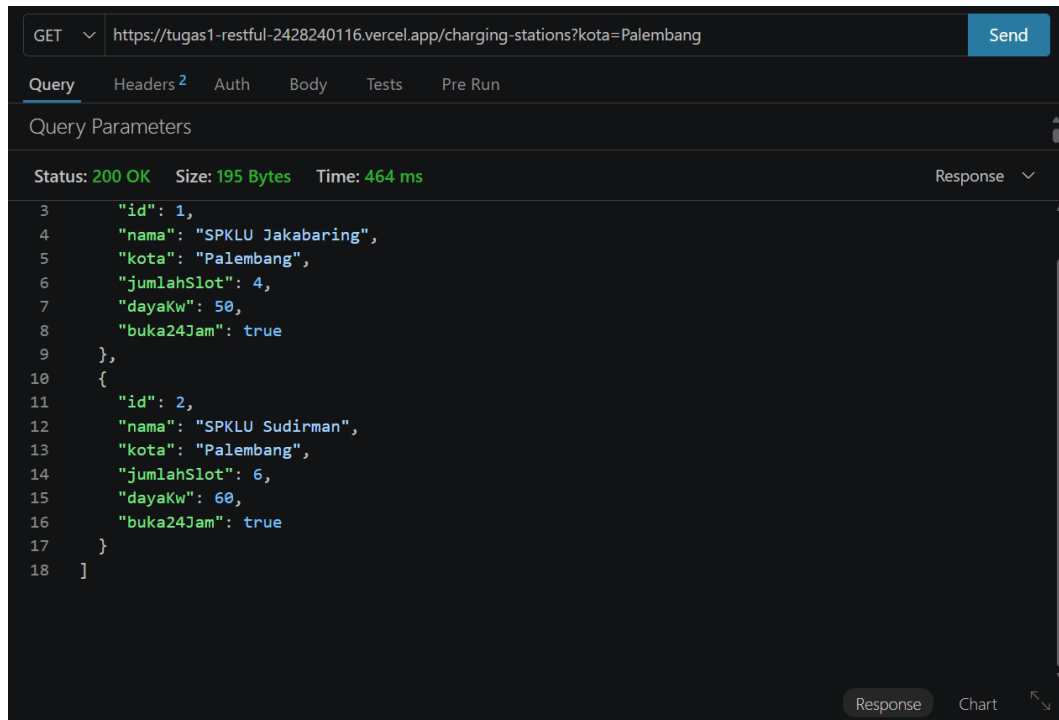
404 Not Found - ID tidak ditemukan



Gambar 5. Bukti Thunder Client untuk GET /charging-stations/99; hasil 404 Not Found - ID tidak ditemukan.

Bukti Pengujian 4: GET /charging-stations?kota=Palembang

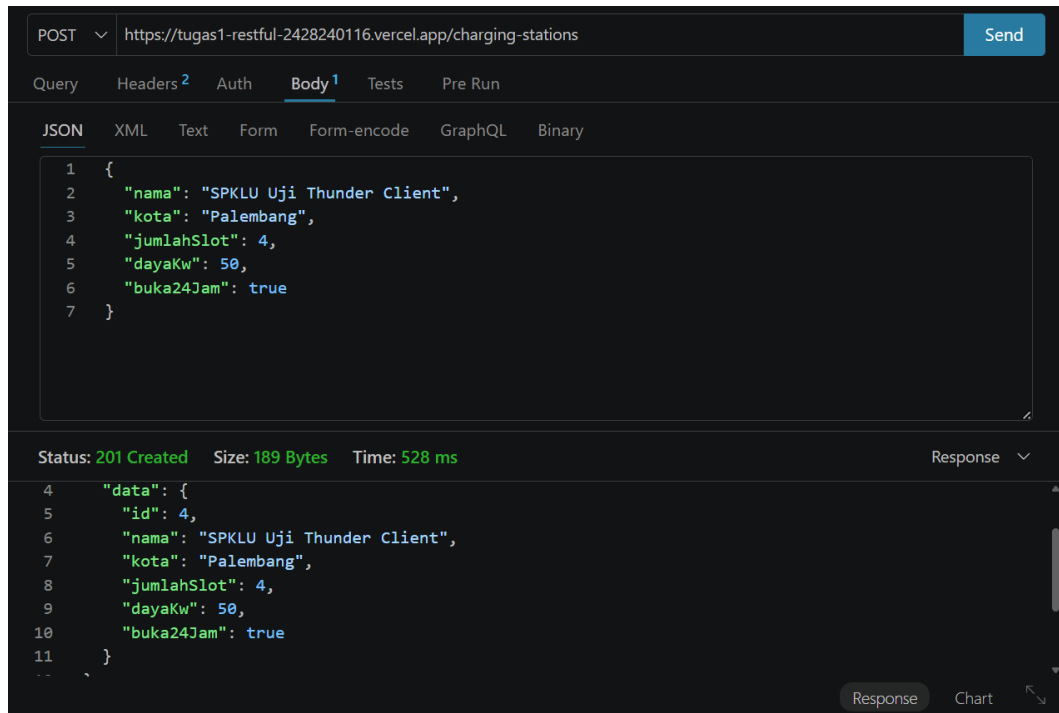
200 OK - filter kota Palembang



Gambar 6. Bukti Thunder Client untuk GET /charging-stations?kota=Palembang; hasil 200 OK - filter kota Palembang.

Bukti Pengujian 5: POST /charging-stations

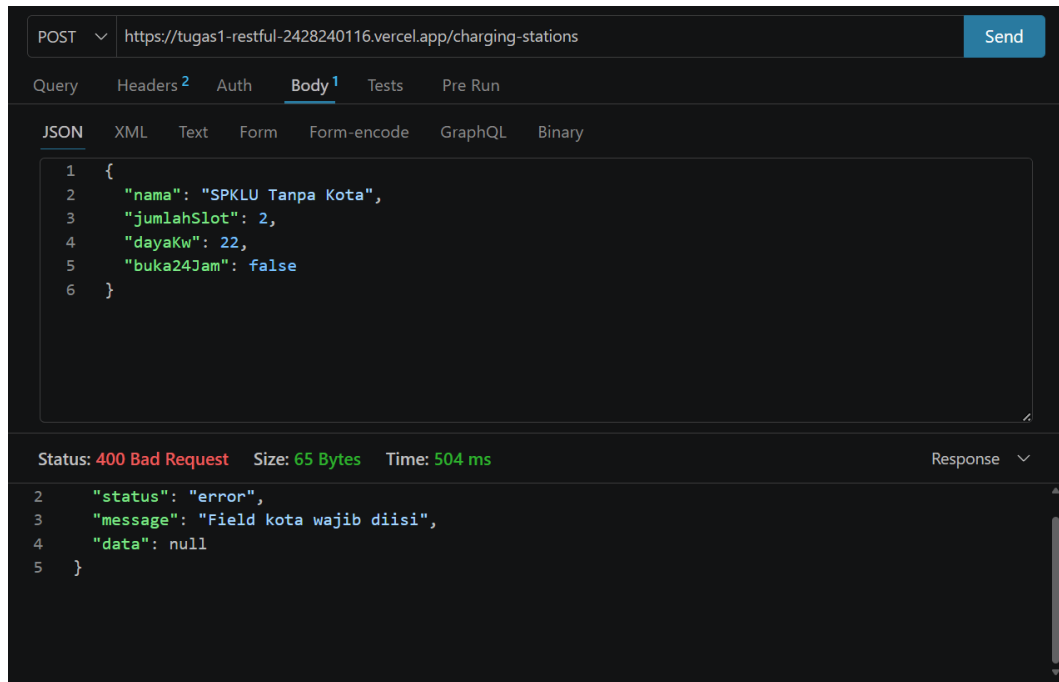
201 Created - data berhasil ditambahkan



Gambar 7. Bukti Thunder Client untuk POST /charging-stations; hasil 201 Created - data berhasil ditambahkan.

Bukti Pengujian 6: POST /charging-stations (tanpa field kota)

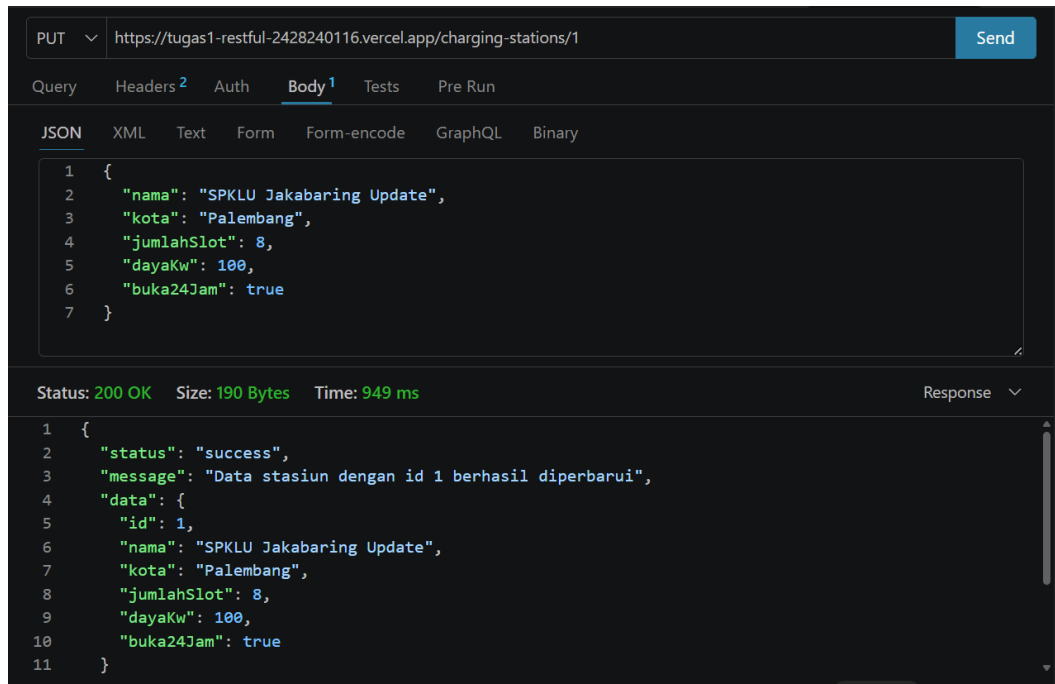
400 Bad Request - validasi field wajib



Gambar 8. Bukti Thunder Client untuk POST /charging-stations (tanpa field kota); hasil 400 Bad Request - validasi field wajib.

Bukti Pengujian 7: PUT /charging-stations/1

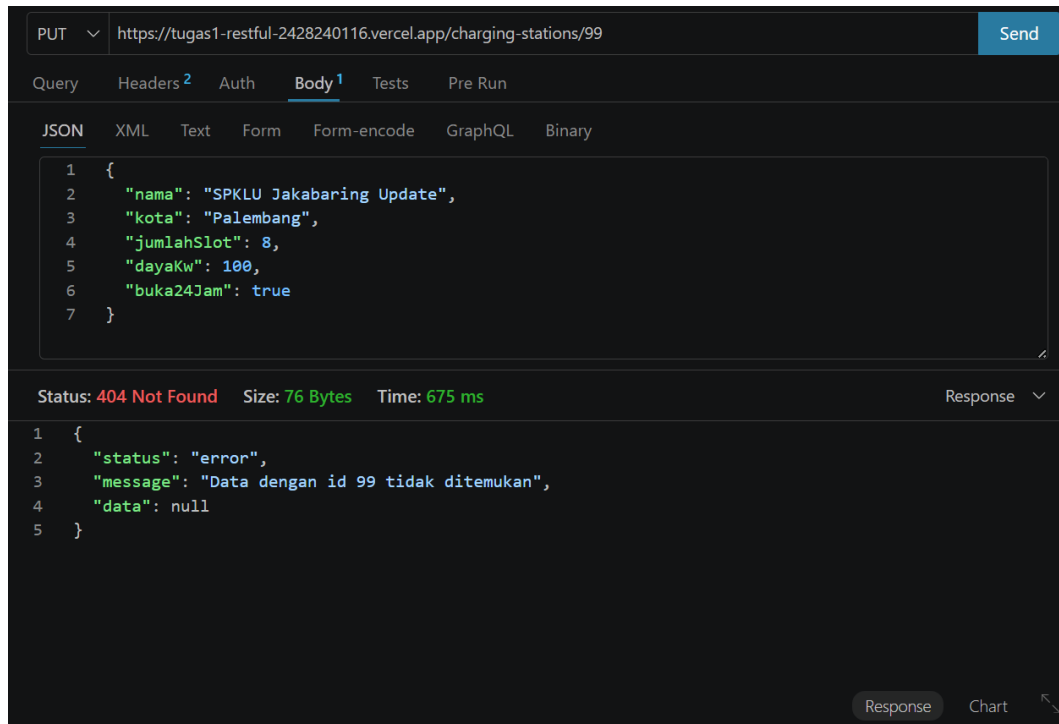
200 OK - data berhasil diperbarui



Gambar 9. Bukti Thunder Client untuk PUT /charging-stations/1; hasil 200 OK - data berhasil diperbarui.

Bukti Pengujian 8: PUT /charging-stations/99

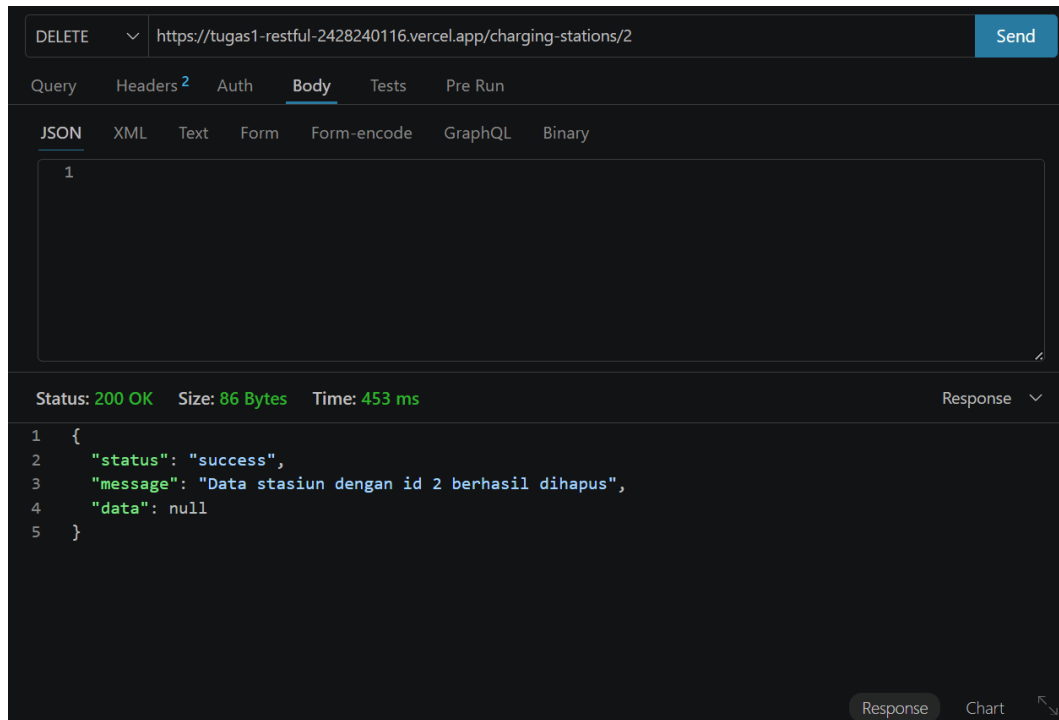
404 Not Found - ID tidak ditemukan



Gambar 10. Bukti Thunder Client untuk PUT /charging-stations/99; hasil 404 Not Found - ID tidak ditemukan.

Bukti Pengujian 9: DELETE /charging-stations/2

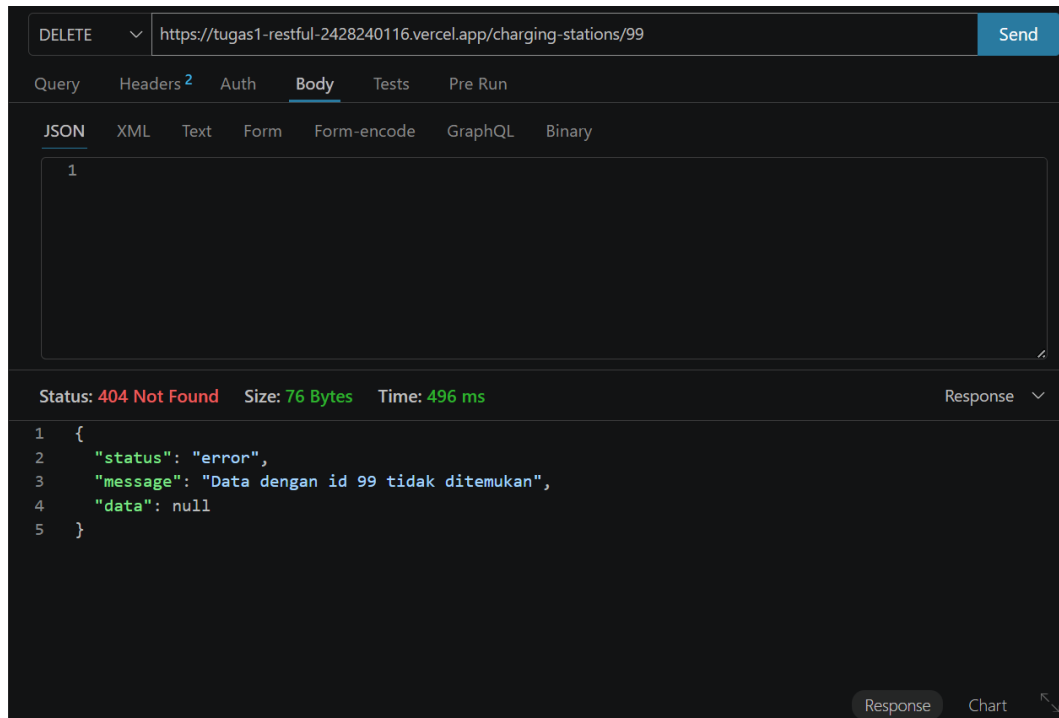
200 OK - data berhasil dihapus



Gambar 11. Bukti Thunder Client untuk DELETE /charging-stations/2; hasil 200 OK - data berhasil dihapus.

Bukti Pengujian 10: DELETE /charging-stations/99

404 Not Found - ID tidak ditemukan



Gambar 12. Bukti Thunder Client untuk DELETE /charging-stations/99; hasil 404 Not Found - ID tidak ditemukan.

VI. Kendala dan Solusi

No.	Kendala	Solusi
1	npm init -y dijalankan pada C:\ dan muncul EPERM.	Project dipindahkan ke folder pengguna yang dapat ditulis, kemudian perintah dijalankan dari folder project.
2	Terminal pernah berada pada folder project yang salah sehingga file tidak ditemukan dan muncul EJSONPARSE.	Lokasi kerja diperiksa melalui Get-Location dan dir; kode JavaScript ditempatkan pada app.js, sedangkan package.json tetap berformat JSON.
3	Request PUT menggunakan curl.exe sempat mengirim body JSON yang tidak valid dari PowerShell.	Body dibuat dengan ConvertTo-Json - Compress dan dikirim memakai Invoke-RestMethod.
4	Data array di Vercel tidak dijamin bertahan pada request selanjutnya karena model serverless.	Pengujian memeriksa respons tiap request; perilaku array in-memory dicatat sebagai batasan sesuai instruksi tugas.

VII. Kesimpulan

RESTful API untuk resource stasiun pengisian kendaraan listrik berhasil dibuat menggunakan Express.js dengan enam endpoint yang disyaratkan. Pengujian melalui Thunder Client pada deployment Vercel menunjukkan status HTTP sesuai untuk operasi berhasil dan gagal, termasuk validasi 400 dan data/endpoint tidak ditemukan 404.

Daftar Pustaka

Express.js. (n.d.). Routing. <https://expressjs.com/en/guide/routing.html>

Mozilla. (n.d.). HTTP request methods. MDN Web Docs.

<https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Methods>

Tim Pengampu Pengembangan Aplikasi Web II. (2026). Tugas 1 - RESTful API Murni dengan Express.js (SI5B) [Dokumen instruksi praktikum]. Universitas Multi Data Palembang.

Vercel. (n.d.). Express on Vercel. <https://vercel.com/docs/frameworks/backend/express>

Lampiran

Tautan Repository GitHub:

GitHub: <https://github.com/naufal20102006/tugas1-restful-2428240116>

Tautan Deployment (jika ada):

Vercel: <https://tugas1-restful-2428240116.vercel.app>
